

Homework 6

CS/ECE 374-B Intro to Algorithms & Models of Computation

University of Illinois Urbana-Champaign

Spring 2025

Instructors: Abhishek Umrawal & Ivan Abraham

Maximum Points: 100

Due Date: Friday, April 11, 2025, 11:59 PM

Instructions

1. Answer **ALL** questions.
2. Show all the work.
3. Legibly scribe your solutions and upload them as individual problems on Gradescope.

Suggestions

While writing your answers and/or proofs, think about the following.

1. What is it that you are given? What is it that you are assuming? What is it that you are trying to prove?
2. How exactly does a step follow from the previous step? Assume nothing is apparent, even algebraic simplifications. Your reasoning should be clear and convincing.
3. If you are using a result from the class then state it clearly before using it. You should identify where exactly you need that result, cast your problem into the setup of the result, and then use it.
4. If you are using a result from outside the class then you should prove it before using it.

While writing and analyzing an algorithm, note the following.

1. An algorithm is a finite set of primitive and unambiguous instructions to solve a computing problem.
2. Unless otherwise stated, you should give the fastest possible algorithm, i.e., an algorithm with the tightest possible asymptotic time complexity bound.

Instructions while using Dynamic Programming

While using dynamic programming to solve a problem, note the following.

1. Your solution must include a *recurrence* relation.
2. Your solution must include a brief description of what your recurrence relation and each of its cases represent.
3. Your solution must include a brief description of the *memory element/storage* and how it is filled in.
4. Your solution must include a *runtime* analysis.
5. Any recursive or dynamic programming algorithm without an English description of the recursive function, i.e., a description of the output of the function in terms of its inputs, will receive **no** credit.
6. **Greedy algorithms require proofs of correctness.** If you choose to design a greedy algorithm, you must prove that your algorithm is correct, or you will receive no credit, despite your algorithm being correct. This means you should check: ‘Are you absolutely sure you do not want to use dynamic programming instead?’

Problems

1. Given an $n \times m$ rectangle (*matrix*) M where each cell is assigned a positive integer value, *cut* M into several rectangles such that all cells in a single rectangle have the same value, following some rules:
 - i. A rectangle can be cut only along one of its horizontal or vertical grid lines, splitting it into two rectangles.
 - ii. Cut one rectangle at a time.

The cost of cutting the rectangle is the *area of the rectangle*—the number of cells in it.

Design and analyze an algorithm to compute the *minimum* cost of cutting an $n \times m$ rectangle M , following the above rules.

Hint: Two points on the same diagonal line can be used to represent a rectangle. For example, $(0,0)$ and $(5,3)$ represent the rectangle in Figure 1.

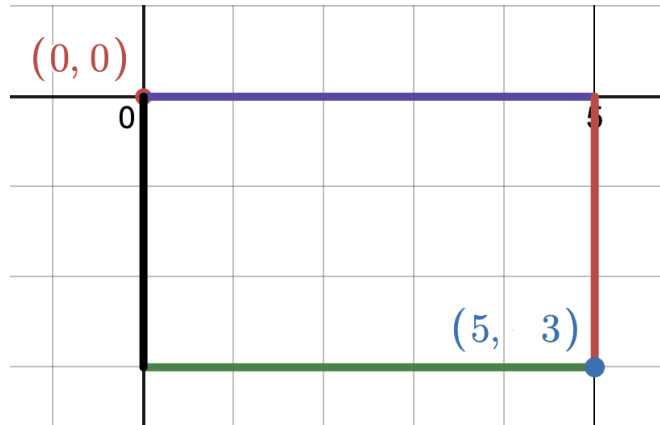


Figure 1: An example rectangle.

Solution: We first introduce a sub-problem defined by a rectangle, such a rectangle is determined by two points $p, q \in M = [n] \times [m]$. We first precompute, for each $p, q \in M$, if the rectangle,

$$r(p, q) = \{(i, j) \in M \mid x(p) \leq i \leq x(q) \text{ and } y(p) \leq j \leq y(q)\}$$

is valid—that is, the labels for all cells in $r(p, q)$ are the same.

Introduce the function `label(p, q)` which returns the label in the rectangle r , denoted with p as the top left corner and q as the bottom right corner, if all cells in it have the same label; and it will return BAD if the labels are different. Then we have the recurrence given in Algorithm 1.

Algorithm 1 `label(p, q)`

```

1: if  $p = q$  then
2:   return  $M[p]$ 
3: if  $x(p) < x(q)$  then
4:    $mm \leftarrow \lfloor (x(p) + x(q))/2 \rfloor$ 
5:    $l_1 \leftarrow \text{label}(p, (mm, y(q)))$ 
6:    $l_2 \leftarrow \text{label}((mm + 1, y(p)), q)$ 
7: else
8:    $mm \leftarrow \lfloor (y(p) + y(q))/2 \rfloor$ 
9:    $l_1 \leftarrow \text{label}(p, (x(q), mm))$ 
10:   $l_2 \leftarrow \text{label}((x(p), mm + 1), q)$ 
11: if  $l_1 \neq l_2$  then
12:   return BAD
13: return  $l_1$ 

```

The function `label(p, q)` contains at most $\mathcal{O}((nm)^2)$ different recursive calls. So it takes $\mathcal{O}((nm)^2)$ time to fill the DP table, of size $\mathcal{O}((nm)^2)$. And the filling order will be starting from the rectangle with size $1, 2, 3, \dots, nm$. Assume V is the memoization structure we use, $V[p, q]$ will store the value of `label(p, q)`.

Next, let us also define a helper function (given in Algorithm 2) that runs in $\mathcal{O}(1)$ time to calculate the price of making one cut of a given rectangle.

Algorithm 2 `price(p, q)`

```

1: if  $x(p) > x(q)$  or  $y(p) > y(q)$  then
2:   return  $\infty$ 
3: return  $(x(p) - x(q) + 1) \cdot (y(q) - y(p) + 1)$ 

```

The solution is now straightforward. Given a rectangle, we should try all possible cuts and take the minimum of these choices. The base case is the time when the current rectangle has all the same labels, which could be determined by V in $\mathcal{O}(1)$ time. Denote the function `best-cut(p, q)` that will return the minimum cost to cut the rectangle given by $M[p(x), \dots, q(x)]$, $[p(y), \dots, q(y)]$ given in Algorithm 3.

Algorithm 3 `best-cut(p, q)`

```

1: if  $x(p) > x(q)$  or  $y(p) > y(q)$  then
2:   return  $\infty$ 
3: if  $V[p, q] \neq \text{BAD}$  then
4:   return 0
5:  $\rho \leftarrow \infty$ 
6: for  $x = x(p) + 1$  to  $x(q)$  do
7:    $c \leftarrow \text{price}(p, q) + \text{best-cut}(p, (x - 1, y(q))) + \text{best-cut}((x, y(p)), q)$ 
8:   if  $c < \rho$  then
9:      $\rho \leftarrow c$ 
10: for  $y = y(p) + 1$  to  $y(q)$  do
11:    $c \leftarrow \text{price}(p, q) + \text{best-cut}(p, (x(q), y - 1)) + \text{best-cut}((x(p), y), q)$ 
12:   if  $c < \rho$  then
13:      $\rho \leftarrow c$ 
14: return  $\rho$ 

```

The function `best-cut`(p, q) contains at most $\mathcal{O}((nm)^2)$ different recursive calls. So it takes $\mathcal{O}((nm)^2)$ time to fill the DP table, of size $\mathcal{O}((nm)^2)$. And the filling order will be starting from the rectangle with size $1, 2, 3, \dots, nm$. The initial function call or the return value from the DP structure will be `best-cut`((0, 0), (n, m)). Since each recursive call takes $\mathcal{O}(n + m)$ time, the total runtime will be $\mathcal{O}((nm)^2(n + m))$.

2. Define a box in three dimensions as an *axis-aligned* product of three closed intervals. To be more precise, a box $B = [x_1, x_2] \times [y_1, y_2] \times [z_1, z_2]$ is defined by three pairs of coordinates, much like a rectangle is defined by two pairs. We say a pair of boxes (B, B') is *fitted* if $B \subset B'$ or $B' \subset B$. In other words, a pair of boxes is *fitted* if we can place one box from the pair in or over the other box from the pair. Given a set $\mathcal{B}$ of n boxes, design and analyze an efficient algorithm to compute the *largest fitted subset* $\mathcal{S} \subseteq \mathcal{B}$. A set of boxes $\mathcal{S}$ is said to be *fitted* if *each* pair of boxes in it is fitted.

Solution: Build a DAG, where $i \rightarrow j$ is an edge in the graph if $B_i \subset B_j$ (B_i fits in B_j), where B_i is the i th box in the input. This graph takes $O(n^2)$ time to construct. This graph does not contain cycles, and is thus a DAG. Then, we just need to find the longest path in a DAG using topological sort, and that can be done in linear time in the size of the DAG, which is $O(n^2)$ in this case.

3. We are given a list $D[1, \dots, n]$ of n words each of length k over an alphabet Σ . Furthermore, $D[1, \dots, n]$ is sorted in lexicographic order. Using $D[1, \dots, n]$ as input, design and analyze an efficient algorithm to obtain the *ordered alphabet* Σ_{ordered} defined as a *permutation* of the symbols in Σ in the *order* they appear in the words in $D[1, \dots, n]$. For example, given an alphabet $\Sigma = \{Q, X, Z\}$ and the list $D[1, \dots, 4] = \{QQZ, QZZ, XQZ, XQX, XXX\}$, the algorithm should return $\Sigma_{\text{ordered}} = \{Q, Z, X\}$. You may assume that D always contains enough information (words) to determine the order of the symbols in the entire alphabet.

Hint: Model the problem as a *graph* where each node represents a letter from the alphabet.

Solution: The key idea is to exploit the lexicographic ordering given by D to deduce relative precedence among the symbols in Σ . We achieve this by comparing consecutive words in D and identifying the first position where they differ; this difference produces a precedence constraint between the two differing letters. We represent these constraints in a directed graph G (with vertices corresponding to symbols in Σ), where an edge from a to b indicates that a precedes b . Finally, a topological sort on G yields the desired order of the alphabet.

• **Graph Construction:**

- (a) Initialize an empty directed graph G and a mapping `inDegree` for all symbols in Σ (initially, `inDegree[a] = 0` for every $a \in \Sigma$).
- (b) For each consecutive pair of words $D[i]$ and $D[i + 1]$, compare the words character by character. Let j be the first index where $D[i][j] \neq D[i + 1][j]$. Add a directed edge from $D[i][j]$ to $D[i + 1][j]$ in G (if not already present) and increment `inDegree[D[i + 1][j]]`.

• **Topological Sort:**

- (a) Initialize a queue L with all symbols having `inDegree = 0`.
- (b) While L is not empty, dequeue a symbol a and append it to the result string. For each neighbor b of a in G , decrement `inDegree[b]`. If `inDegree[b]` becomes 0, enqueue b .

- The resulting string from the topological sort is the order of the alphabet.

Algorithm 4 BUILDGRAPH ($D[1 \dots n]$)

```

1: Initialize a graph  $G$  with a vertex for every symbol in  $\Sigma$ 
2: Set inDegree[a] ← 0 for every  $a \in \Sigma$ 
3: for  $i = 1$  to  $n - 1$  do
4:   word1 ← D[i], word2 ← D[i + 1]
5:   for  $j = 1$  to  $\min(|\text{word1}|, |\text{word2}|)$  do
6:     if word1[j] ≠ word2[j] then
7:       if word2[j] is not in  $G[\text{word1}[j]]$  then
8:         Add a directed edge from word1[j] to word2[j] in  $G$ 
9:         inDegree[word2[j]] ← inDegree[word2[j]] + 1
10:      break ▷ Only the first differing character matters
11: return ( $G$ , inDegree)
```

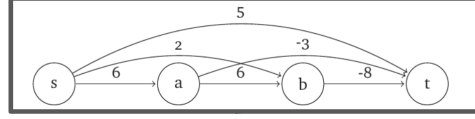
Algorithm 5 FINDALPHABETORDER ($D[1 \dots n]$)

```

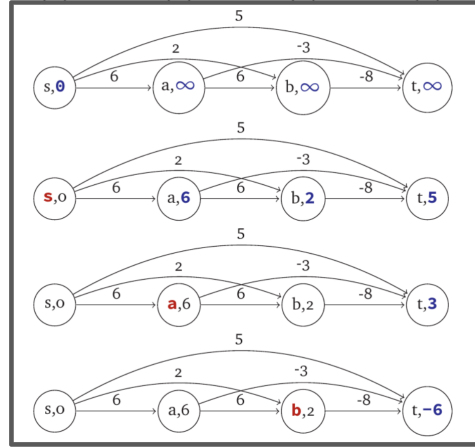
1: ( $G$ , inDegree) ← BUILDGRAPH( $D$ )
2: alphabetOrder ← TopSort(G) ▷ Call TopSort
3: if length(alphabetOrder) ≠ |\Sigma| then
4:   return "Cycle detected: Invalid input"
5: else
6:   return alphabetOrder
```

4. Given a *directed acyclic graph*, $G = (V, E)$, with integer (positive, negative, or zero) edge weights, design and analyze an algorithm to find the *shortest* path from a node $s \in V$ to a node $t \in V$.

Solution: Because there are negative edge weights, we cannot use a greedy method like Dijkstra's algorithm; however, because the graph is directed and acyclic, we can use a topological sort. Topologically sorting the graph means that every vertex can only reach vertices below it in the sort and cannot reach vertices above it in the sort.



This means, after topologically sorting the graph, we start at node s and compute the shortest path from node s to each of the nodes below it in sequential order. To do this, we initialize the distance from s to all the other nodes as ∞ , then we update the distance from node s to be the weight of the edges from s to the neighbors of s . Then we move on to the next sequential node, say u , and look at its neighbor, say v . If the distance from s to u plus the edge weight from u to v is less than the current distance from s to v , then we update the value. This repeats until we reach node t .



Let s and t be the n th and m th node in the topological sort, and let $N(u)$ be the neighbor set of u , the required algorithm is given in Algorithm 6.

Algorithm 6 DAGSP(V, E, s, t):

```

1:  $(Vs, Es) \leftarrow TopSort(V, E)$ 
2:  $n \leftarrow index(s)$ 
3:  $m \leftarrow index(t)$ 
4:  $D[n] \leftarrow 0$ 
5: for  $k \leftarrow n + 1$  to  $m$  do
6:    $D[k] \leftarrow \infty$ 
7:   for  $i \leftarrow n$  to  $m - 1$  do
8:     for  $v$  in  $N(v_i)$  do
9:        $j \leftarrow index(v)$ 
10:       $D[j] \leftarrow \min\{D[j], D[i] + Es[i][j]\}$ 
11: Return  $D[m]$ 

```

A topological sort takes $O(|V| + |E|)$ time, and the for loops in the algorithm take $O(|V| + |E|)$. So the total running time is $O(|V| + |E|)$.

5. Design and analyze an efficient algorithm to compute the maximum points earned in a **Number Blast** puzzle of size n described as follows:

- i. The puzzle starts with an array of $2n$ numbered blocks.
- ii. On each turn, you select two *adjacent blocks*—blocks with no other blocks between them
- iii. You earn points equal to the product of the numbers of the two chosen blocks, and in each turn, both the chosen blocks are removed.
- iv. Once a block is removed, it cannot be chosen in future turns.
- v. After a turn, the remaining blocks are concatenated together for the next turn.
- vi. The process continues to repeat until all blocks are removed.

The goal is to reach the end of the puzzle, i.e., all blocks are removed, such that the total points earned is maximized.

Hints: i) Think about the last move. ii) An algorithm with a runtime of $O(n^3)$ can be designed using DP + Memoization, but other correct algorithms with a runtime of $O(n^4)$ will also qualify for full credit.

Solution: A key observation is that every valid sequence of moves corresponds to a **non-crossing pairing** of the $2n$ numbers. If we label the blocks as $a_1, a_2, \dots, a_{2n}$, then a valid removal sequence defines a pairing of these indices such that no two pairs “cross” when drawn above the sequence. Moreover, given any non-crossing pairing, one can always remove the squares in an order (for example, by repeatedly removing the innermost pair) so that the total score is the sum of the products of the paired numbers.

Define $\text{NumberBlast}(i, j)$ as the maximum score obtainable from the subarray $a_i, a_{i+1}, \dots, a_j$, where the number of elements $j - i + 1$ is even. The optimal solution for the full puzzle is then $\text{NumberBlast}(1, 2n)$.

- **Base Case:** When the subarray has no elements (i.e., $i \geq j$), there is no possible move:

$$\text{NumberBlast}(i, j) = 0, \forall i \geq j.$$

- **Recurrence:** For a subarray with more than two elements, consider pairing the first element a_i with some later element a_k , where k takes values $i+1, i+3, \dots, j$ (this guarantees that the subarray between a_i and a_k contains an even number of elements and thus can be completely removed). When a_i is paired with a_k , the total score is the sum of:
 - (a) The score from pairing a_i and a_k , which is $a_i \cdot a_k$.
 - (b) The maximum score from removing the subarray *between* a_i and a_k , i.e., $\text{NumberBlast}(i+1, k-1)$.
 - (c) The maximum score from removing the subarray *after* a_k , i.e., $\text{NumberBlast}(k+1, j)$.

Thus, the recurrence is given by:

$$\text{NumberBlast}(i, j) = \begin{cases} 0, & \forall i \geq j, \\ \max \left\{ \begin{array}{l} a_i \cdot a_k \\ +\text{NumberBlast}(i+1, k-1) \\ +\text{NumberBlast}(k+1, j) \end{array} \middle| \begin{array}{l} k-i \text{ is odd, } i < k \leq j \end{array} \right\}, & \text{otherwise.} \end{cases}$$

Finally, we analyze the time and space complexity of this dynamic programming algorithm.

- **Time Complexity:** There are $O((2n)^2) = O(n^2)$ subproblems. For each subproblem, we consider up to $O(n)$ choices for k . Hence, the overall time complexity is $O(n^3)$.
- **Space Complexity:** The dynamic programming table requires $O(n^2)$ space.

Algorithm 7 Dynamic programming solution of Number Blast with memoization

```
1: Global: A 2D array memo initialized with undefined values.
2: procedure COMPUTEMAXIMUMSCORE
3:   Initialize the 2D array memo for indices  $1 \leq i, j \leq 2n$  with all values set to undefined
4:   return NUMBERBLAST(1,  $2n$ )
5:
6: procedure NUMBERBLAST( $i, j$ )
7:   if  $i \geq j$  then
8:     return 0
9:   if memo[ $i$ ][ $j$ ] is defined then
10:    return memo[ $i$ ][ $j$ ]
11:    $best \leftarrow -\infty$ 
12:   for  $k \leftarrow i + 1$  to  $j$  do
13:     if  $(k - i)$  is odd then
14:        $score \leftarrow a[i] \times a[k] + \text{NUMBERBLAST}(i + 1, k - 1) + \text{NUMBERBLAST}(k + 1, j)$ 
15:        $best \leftarrow \max(best, score)$ 
16:   memo[ $i$ ][ $j$ ]  $\leftarrow best$ 
17:   return  $best$ 
```
